Week 2 Database Design Report

Intern: Abdullah Aslam
Track: Software Development
Project: Student Records Manager
Repository: https://github.com/abdullahkjn2812-web/student-records-manager
Date: 21 October 2026

1. Week 2 objective

Identify the data the application must store, design MongoDB entities with keys and relationships, produce an ER diagram, and set up the initial Express + Mongoose backend skeleton. Student CRUD, authentication, and dashboards are intentionally left for later weeks.

2. Technology stack

Layer	Choice
Backend	Node.js and Express.js
Database	MongoDB
ODM	Mongoose
Environment	dotenv
Frontend (later)	React.js + Vite
API testing (later)	Postman
Version control	Git + GitHub

3. Entities and fields

User (collection: users)

Independent staff account used later for login. No teacher-to-student assignment is modelled in Week 2.

Field	Type	Rules
_id	ObjectId	Primary identifier, generated by MongoDB
name	String	Required
email	String	Required, unique, stored lowercase
password	String	Required; hashing is deferred to Week 4
role	String	Enum: admin or teacher (default teacher)
createdAt / updatedAt	Date	Mongoose timestamps

Student (collection: students)

Field	Type	Rules
_id	ObjectId	Primary identifier
name	String	Required
rollNumber	String	Required, unique
className	String	Required
email	String	Optional
phone	String	Optional
createdAt / updatedAt	Date	Mongoose timestamps

Marks (collection: marks)

Field	Type	Rules
_id	ObjectId	Primary identifier
student	ObjectId	Required reference to Student
subject	String	Required, trimmed and lowercased
obtainedMarks	Number	Required, finite, 0 to totalMarks
totalMarks	Number	Required, finite, greater than 0
createdAt / updatedAt	Date	Mongoose timestamps

A compound unique index on `(student, subject)` prevents two results for the same student and subject.

4. Database relationships

Student → Marks: one-to-many.

Each marks document stores the student's `_id` in `Marks.student`. MongoDB does not create a SQL-style foreign key. Mongoose uses an ObjectId plus `ref: "Student"` so later code can `populate()` the student. This relationship is needed because one student has several subject results, while each result belongs to exactly one student.

User is independent in Week 2. No extra relationships were added.

5. ER diagram

The diagram is saved as `docs/ER_Diagram.drawio` (editable) and `docs/ER_Diagram.png` (export).



6. Initial backend architecture

File	Purpose
server/server.js	Loads env, defines /health, starts HTTP only after MongoDB connects
server/config/db.js	Connects Mongoose and initialises model indexes
server/models/User.js	User schema
server/models/Student.js	Student schema
server/models/Marks.js	Marks schema, range check, unique student+subject index
server/routes and controllers	Placeholders for Week 3 CRUD
server/.env.example	Sample MONGO_URI and PORT without secrets
client/	Reserved for the React app

The Express server binds to 127.0.0.1. A missing or invalid MongoDB URI stops startup. `.env` is gitignored.

7. Database setup

1. Copy `server/.env.example` to `server/.env`.
2. Set `MONGO_URI` to a local MongoDB database (default `student_records_manager`).
3. From `server/` run `npm install` then `npm start`.
4. Confirm `GET /health` returns `status: "ok"` and `database: "connected"`.

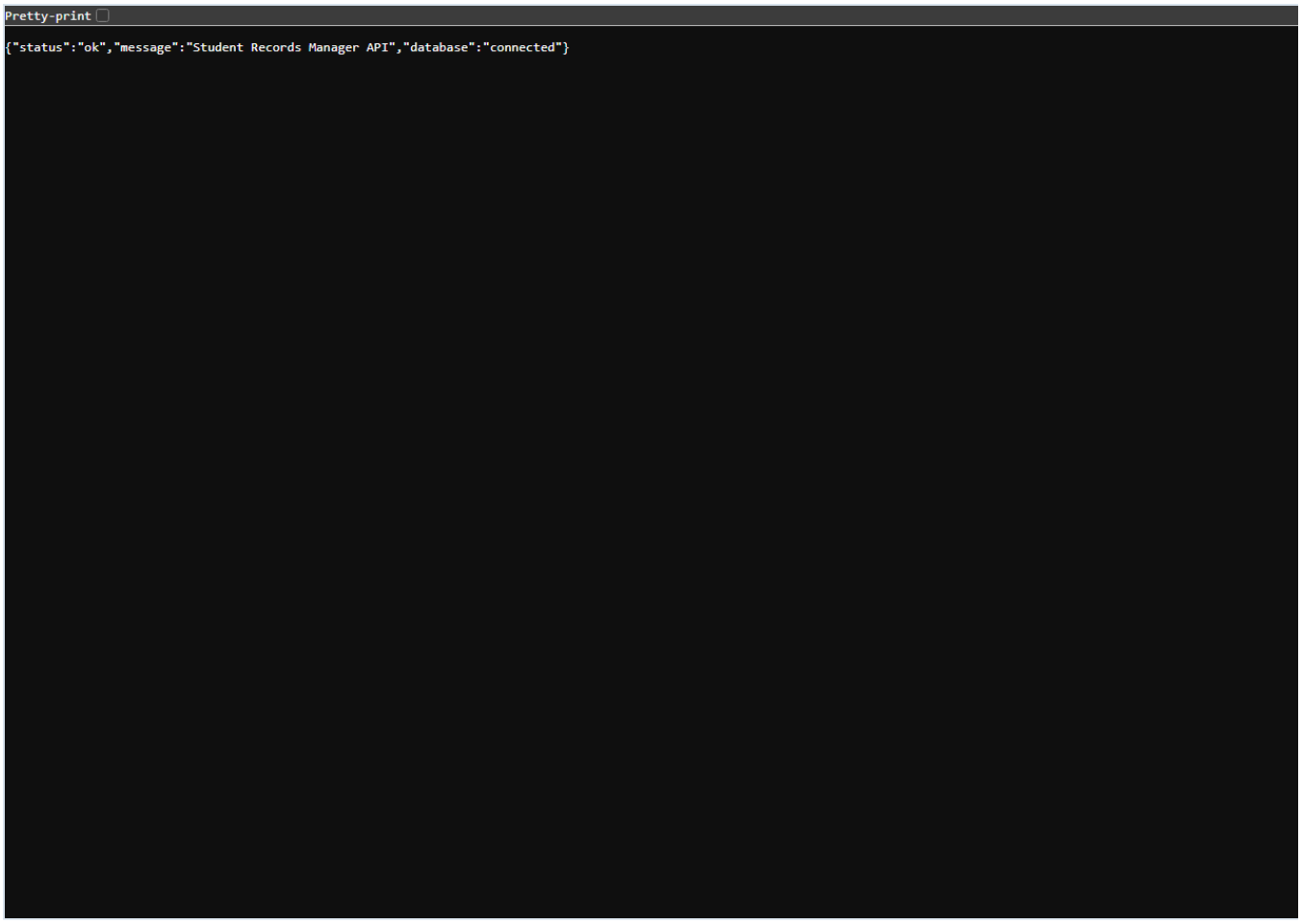
8. Verification results

These results were produced on 21 October 2026 against a running local MongoDB Windows service (`mongod` process active).

Check	Command	Result
Model validation	`node --test tests/models.test.js`	27 passed, 0 failed
Integration	`npm run test:integration` with `student_records_manager_test_week2`	5 passed, 0 failed
Backend start	`node server.js`	`MongoDB connected: student_records_manager` then listening on 127.0.0.1:5000
Health endpoint	`GET http://127.0.0.1:5000/health`	`{"status":"ok","message":"Student Records Manager API","database":"connected"}`
Collections	Mongoose `listCollections()`	`users`, `students`, `marks` with unique email, rollNumber, and student+subject indexes
.env not tracked	`git check-ignore -v server/.env`	`.gitignore:4:.env`
Week 1 documents	`git ls-files`	Project_Proposal.pdf and Requirements_Document.pdf still tracked

MongoDB Compass was not installed on this computer. The live database was inspected through Mongoose instead of a Compass screenshot.

Health endpoint (browser):



Startup, MongoDB collections, tests and folder structure:

Week 2 live verification — 21 October 2026

Captured from the local machine after MongoDB and Express started. Not a mock.

1. Backend startup and MongoDB connection

```
command: node server.js
cwd: server/

MongoDB connected: student_records_manager
Server listening on http://127.0.0.1:5000
```

2. GET /health

```
command: Invoke-RestMethod http://127.0.0.1:5000/health

{"status":"ok","message":"Student Records Manager API","database":"connected"}
```

3. MongoDB collections and indexes (Mongoose)

```
database: student_records_manager
collections: students, users, marks

students indexes: _id_, unique rollNumber_1
users indexes: _id_, unique email_1
marks indexes: _id_, unique student_1_subject_1

MongoDB Compass was not installed. Collections were inspected through Mongoose against the running mongod service.
```

4. Model tests

```
command: node --test tests/models.test.js

# tests 27
# pass 27
# fail 0
```

5. Integration tests

```
command: npm run test:integration
TEST_MONGO_URI=mongodb://127.0.0.1:27017/student_records_manager_test_week2

ok 1 unique user emails and timestamps
ok 2 unique student roll numbers
ok 3 student-to-marks one-to-many and no duplicate subject
ok 4 /health 200 and /students 404
ok 5 backend does not start if MongoDB is unreachable

# tests 5
# pass 5
# fail 0
```

6. Git ignore and Week 1 documents

```
git check-ignore -v server/.env
.gitignore:4:.env    server/.env

git ls-files docs/Project_Proposal.pdf docs/Requirements_Document.pdf
docs/Project_Proposal.pdf
docs/Requirements_Document.pdf
```

7. Project folder structure

```
student-records-manager/
  client/
  docs/
    Project_Proposal.pdf
    Requirements_Document.pdf
    Week_02_Database_Design.pdf
    ER_Diagram.drawio
    ER_Diagram.png
    er-diagram.html
    week2-report.html
    week2-evidence/
  server/
    config/db.js
    models/User.js, Student.js, Marks.js
    controllers/
    routes/
    server.js
    package.json
    .env.example
  tests/
    models.test.js
    integration.test.js
  README.md
  .gitignore
```

9. Conclusion

Week 2 delivers a small, realistic MongoDB design and a working Express skeleton. The models are ready for Week 3 CRUD. Authentication, dashboards, and extra relationships were not implemented.